

量化交易平台 技术问题全解

QMT / Ptrade 常见问题与实战方案

迅投 QMT—恒生 PTrade—Mini-QMT

目 录

- 1. QMT 常见问题 3
 - 1.1 定时与自动化执行 3
 - 1.2 数据获取与处理 4
 - 1.3 策略编写与回测 4
 - 1.4 交易执行与风控 5
 - 1.5 miniQMT 专项问题 5
- 2. Ptrade 常见问题 6
 - 2.1 策略运行终端与服务器特性 6
 - 2.2 Python 环境兼容性 7
 - 2.3 数据与交易差异 7
- 3. 实盘 vs 模拟盘差异对比 8
- 4. 多策略并行与仓位隔离 11
- 附录：快速问题定位表 14

1. QMT 常见问题

1.1 定时与自动化执行

问题类型	具体表现	解决方案
定时启动/关闭	无法实现策略定时自动启停	使用 Windows 任务计划程序+QMT 命令行启动
定时任务调度	盘中定时执行（如每 5 分钟选股）失效	使用 run_daily 或 run_interval，注意交易所时间同步
时间同步	本地时间与交易所时间不一致导致废单	强制使用 context.current_dt 而非 datetime.now()
节假日跳过	非交易日自动运行导致错误	接入交易日历库，启动前判断 is_trade_day
异常重启	断网或崩溃后无法自动恢复	miniQMT 配合 Python 守护进程，异常后自动重连

1.2 数据获取与处理

- **历史数据下载**：日线/分钟线下载失败，需检查本地存储路径权限及磁盘空间
- **实时数据延迟**：tick 推送延迟，优先使用 Level-2 行情；检查网络防火墙是否拦截券商端口
- **复权处理**：除权除息日数据异常，建议统一使用后复权计算信号，前复权展示收益
- **数据存储位置**：默认在 QMT 安装路径/userdata/datacache/，需定期清理避免占满 C 盘

1.3 策略编写与回测

- **未来函数陷阱**：使用当日收盘价计算当日买卖点（如 close[-1]在盘中实际是未来数据）
- **回测实盘差异**：回测能成交但实盘无法成交，需设置合理滑点（建议万 5-万 10）

- **Context 对象**：变量需在 initialize 中初始化，避免重启后状态丢失；使用 g.全局对象传递状态
- **运行速度优化**：避免 for 循环遍历全市场，改用向量化运算或 get_index_stocks 限定标的池

1.4 交易执行与风控

- **下单失败/废单**：价格超出涨跌停限制、非交易时间下单、资金不足
- **委托状态监控**：通过 get_orders()轮询，处理部分成交需拆分逻辑
- **持仓同步**：人工干预后本地与实盘不一致，每日开盘前强制同步 get_positions()
- **涨跌停处理**：需判断 high_limit/low_limit，涨停板禁止买入，跌停板禁止卖出

1.5 miniQMT 专项问题

- **独立运行**：QMT 主界面关闭后，miniQMT 可继续运行（需保持后台进程 XtMiniQmt.exe）
- **xtquant 调用**：外部 Python 需安装 xtquant 库，注意 Python 版本兼容性（建议 3.6-3.8）
- **数据订阅限制**：单账户同时订阅股票数量上限通常为 300-500 只，超出需轮换订阅

2. Ptrade 常见问题

2.1 策略运行终端与服务器特性

问题	现象	根因
上传同步失败	修改代码后终端显示旧版本	浏览器/终端缓存未刷新，需强制刷新或重新登录
状态显示异常	显示“运行中”实际已崩溃	云端进程僵死，需联系券商后台强制终止
服务器重启丢单	17:00-19:00 清算后委托丢失	服务器每日重启，策略状态重置，需持久化存储
断线重连失败	网络恢复后策略数据丢失	context 对象未持久化，重启后以为今天是首日运行

2.2 Python 环境兼容性（关键痛点）

- Python 2.7 遗留问题：**存量策略使用 `print xxx`、`xrange`、`dict.iteritems()` 等 Py2 语法，在新环境报错
- 整数除法陷阱：**`1/2` 在 Py2=0，在 Py3=0.5，计算仓位时务必使用 `from __future__ import division` 或强制转 `float`
- 第三方库限制：**无法 `pip install`，仅能使用内置库（`numpy`、`pandas`、`sklearn`），且版本较旧
- 编码问题：**中文策略名、注释可能导致上传失败，建议统一使用 UTF-8 无 BOM 格式

2.3 数据与交易差异

- 缓存不清理：**`get_price` 缓存文件堆积占满磁盘空间（通常仅分配几百 MB），需定期调用清理接口

- **废单无提示**：委托价格超出涨跌停，终端不弹窗，仅在日志显示“废单”，需主动监控
`get_orders()`返回值
- **成交推送延迟**：云端到本地回报延迟 5-10 秒，高频策略需考虑异步处理
- **资金计算误差**：`get_available_cash()`与实际可买金额有偏差，建议保留安全垫（如只使用 90%资金）

3. 实盘 vs 模拟盘差异对比

3.1 核心差异总览

维度	Ptrade	QMT
撮合机制	云端虚拟撮合，价格达到即成交	本地历史撮合，当前价即成交
排队机制	无视涨停板队列深度，可“插队”	同样无视队列深度
网络因素	模拟/实盘都在云端，无本地差异	实盘受本地网络影响，断线重连是实盘特有
数据延迟	两者都用云端数据，延迟相同	模拟用本地缓存，实盘用实时推送
资金冻结	模拟盘 T+0 可用，无真实冻结	通常也无真实冻结模拟
风控拦截	模拟盘无券商风控拦截	实盘有异常交易监控

3.2 Ptrade 虚实差异详解

云端撮合的“作弊感”（最大痛点）：模拟盘达到委托价即成交，不考虑交易所订单队列深度；涨停板委托可买到（实际你在队尾买不到）。实盘真相：涨停打板时，模拟盘成功率 90%，实盘可能仅 10-30%。资金幻觉：模拟盘卖出后资金立即可用（T+0），可马上再买；实盘受 T+1 交收限制。

3.3 QMT 虚实差异详解

本地数据缓存陷阱：模拟盘使用下载到本地的历史数据（可能是昨日收盘后的数据），`get_price` 获取本地文件。实盘使用券商推送的实时数据流；若本地数据未更新（如早晨 9:00 启动），策略可能使用昨日收盘价而非当日开盘价。

3.4 共通性致命差异

差异类型	模拟盘表现	实盘真相	应对建议
滑点	固定 0.01 元或无滑点	小盘股冲击成本 0.5%-2%	模拟盘按 3-5 倍滑点预估
市场冲击	买 100 万和 1 万成交价相同	大单吃掉多档盘口	限制单票仓位不超过流通盘的 1%
涨跌停处理	涨停价挂单即成交	排队买不到，跌停卖不出	代码层加入队列深度判断
手续费	固定万 3 佣金	最低 5 元佣金、过户费、规费	高频策略精确计算到分
极端行情	无闪崩、封死涨停模拟	跌停板卖不出，风控限制发单	加入异常处理熔断机制

3.5 验证与过渡方案

Ptrade 过渡策略代码示例：

```
# 在模拟盘加入“真实世界惩罚”
if context.run_info.run_type == 'simulation': # 模拟盘标识
    price = price * (1.002) # 强制加 0.2%滑点
    # 模拟排队：涨停买入成功率仅 30%
    if price >= high_limit * 0.999:
        if random.random() > 0.3: # 70%概率模拟买不到
            return
```

通用 Checklist：

- 模拟盘加入涨停买不到、跌停卖不出的逻辑限制
- 设置资金上限，避免无限资金幻觉
- 检查手续费计算，包含最低 5 元、过户费、流量费
- 模拟盘运行至少 20 个交易日，实盘预期收益通常打 7-8 折

- 首次实盘用 1 手或最小资金单位测试委托回报时间和持仓同步

4. 多策略并行与仓位隔离

4.1 基础痛点（未做隔离时）

- **资金争抢**：策略 A 满仓后，策略 B 因可用资金不足导致废单
- **持仓互斥**：策略 A 卖出某股票，策略 B 同时买入，导致“左手倒右手”损耗手续费
- **盈亏混乱**：无法区分每个策略的独立收益曲线
- **风控串扰**：策略 A 止损清仓，误把策略 B 持有的同一只股票也卖掉

4.2 三种隔离方案

方案 A：物理隔离（推荐新手）

实现：向券商申请多个资金账号（子账户），每个策略绑定独立账号；或信用账户+普通账户分离。优点：彻底隔离，互不影响；风控简单。缺点：资金利用率低；QMT 需多开客户端（每电脑限 3-4 个），Ptrade 需申请多个权限。

方案 B：虚拟账户逻辑隔离（主流做法）

核心机制：单账户内用代码层模拟“虚拟子账户”

```
# 虚拟账户管理示例
class VirtualAccount:
    def __init__(self, strategy_id, init_cash):
        self.strategy_id = strategy_id
        self.virtual_cash = init_cash # 分配虚拟资金
        self.virtual_positions = {} # 独立持仓
        self.frozen_cash = 0 # 冻结资金

# 全局资金调度
class CapitalManager:
    def __init__(self, total_cash):
        self.total = total_cash
        self.accounts = {
            'A': VirtualAccount('A', 500000),
            'B': VirtualAccount('B', 500000)
        }
```

关键技术点：

- **可用资金计算**：总资金扣除其他策略已占用虚拟资金+冻结资金
- **持仓合并**：实盘持仓是各策略持仓总和，卖出时需判断其他策略是否持有
- **委托标记**：下单时通过 remark 或 user_id 标记策略来源，成交回报时正确归属
- **数据持久化**：每日收盘后用 pickle/json 保存虚拟账户状态，防止重启丢失

方案 C：标的池隔离（简易版）

逻辑：策略 A 只做沪深 300 成分股，策略 B 只做中证 1000，天然避免持仓冲突。局限：资金仍共享，极端行情下可能资金不足；策略风格受限。

4.3 券商端限制与对策

- **QMT 限制**：官方建议单账户策略≤5 个，否则卡顿；多策略累加委托易触发券商流量风控
- **Ptrade 限制**：云端资源受限，多策略竞争 CPU 时间片，建议拆分到不同子账户
- **对策**：控制调仓频率，或使用 miniQMT 在本地 Python 调度多个独立进程，通过 API 与 QMT 交互

建议：多策略实盘优先选择支持多子账户的券商（如某主账号下挂 10 子账户），这是成本最低的隔离方案。若只能单账户，必须实现虚拟账户系统，否则实盘必然混乱。

附录：快速问题定位表

现象	可能原因	排查方向
模拟盘赚钱实盘亏	未来函数/滑点差异/涨停买不到	检查是否使用当日收盘数据；检查涨跌停处理逻辑
策略显示运行中但无委托	终端僵死/资金不足/标的过滤全部剔除	检查日志；检查可用资金；打印选股结果
重启后仓位错乱	未做本地持久化	实现 pickle 保存虚拟持仓
盘中突然停止	券商服务器维护/本地网络断开/内存溢出	检查时间是否逢 17:00 清算；检查网络；查看任务管理器
废单过多	价格笼子限制/非交易时间/资金不足	检查价格是否在±2%笼子内；检查交易时间逻辑



量化交易平台技术参考文档

适用平台：迅投 QMT（miniQMT）、恒生 Ptrade

文档版本 v1.0 · 2024 年 12 月

券商 API 接口及规则可能更新，请以最新官方文档为准，本文档仅作参考